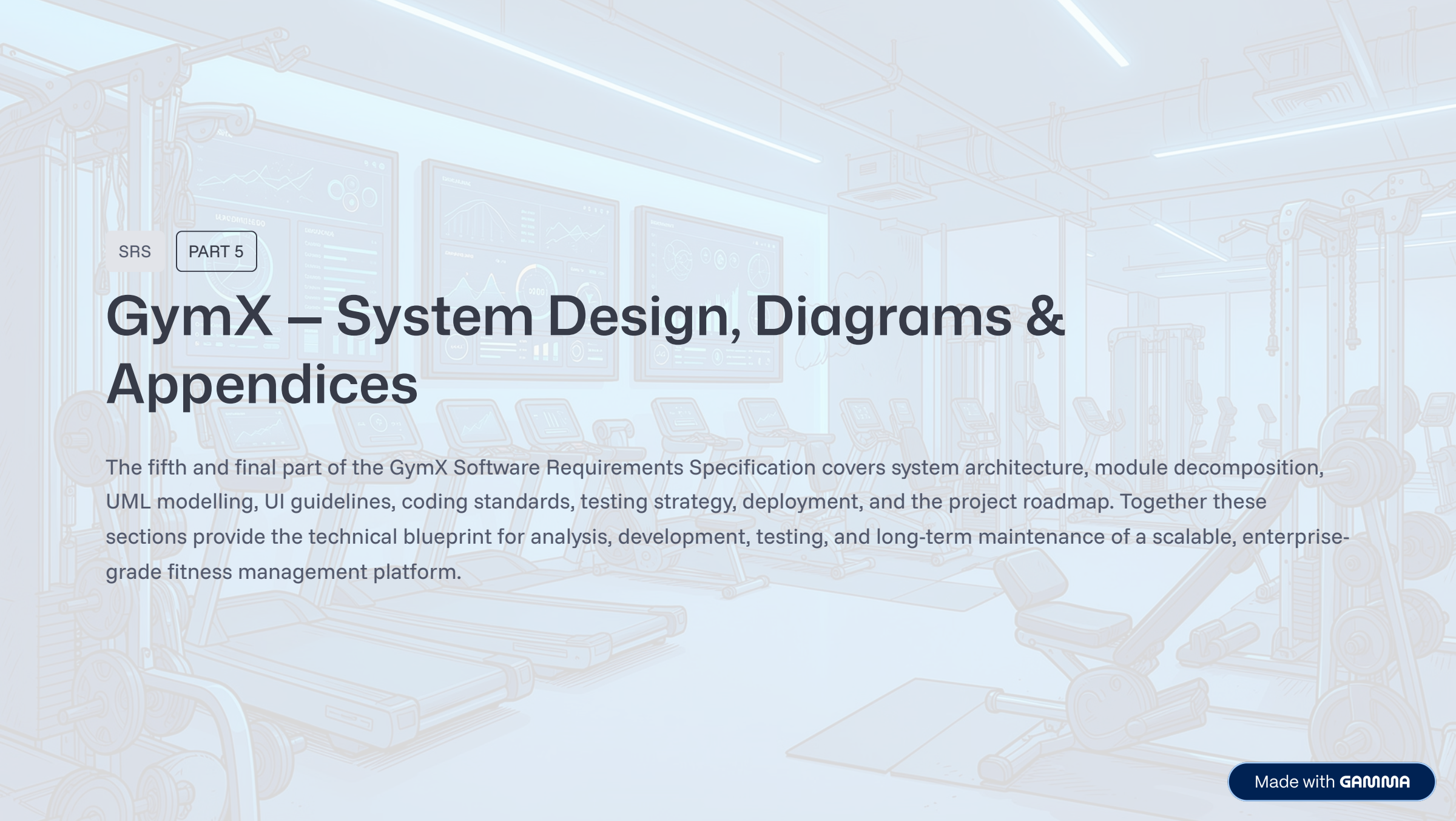




SRS

PART 5

GymX – System Design, Diagrams & Appendices

The fifth and final part of the GymX Software Requirements Specification covers system architecture, module decomposition, UML modelling, UI guidelines, coding standards, testing strategy, deployment, and the project roadmap. Together these sections provide the technical blueprint for analysis, development, testing, and long-term maintenance of a scalable, enterprise-grade fitness management platform.

System Architecture

GymX follows a modern three-tier architecture separating presentation, business logic, and data concerns into distinct layers. This modular approach enables independent scaling, simplifies testing, and ensures the platform remains suitable for future cloud migration without wholesale re-engineering.

Presentation Layer

- React.js / Next.js
- Responsive UI
- Dashboard Interfaces

Business Logic Layer

- Django
- Django REST Framework
- JWT Authentication
- Business Services

Data Layer

- PostgreSQL
- Redis
- File Storage
- Backup Services

The presentation layer delivers a responsive, component-based interface built on React.js with Next.js for server-side rendering and route optimisation. The business logic layer is powered by Django and the Django REST Framework, exposing RESTful APIs secured via JWT authentication. Persistent state resides in PostgreSQL, with Redis providing high-performance caching and session management. File storage and scheduled backup services complete the data tier. Each layer communicates exclusively through well-defined interfaces, ensuring loose coupling and enabling targeted horizontal scaling as demand grows.

System Modules

The GymX platform is composed of twenty-one functional modules that collectively span every operational facet of a modern fitness centre — from member onboarding and billing through equipment maintenance and AI-assisted coaching.



Access & Users

Authentication, User Management, Role & Permission Management, Member Management, Employee Management, Branch Management.



Fitness & Nutrition

Workout Management, Nutrition Management, Class Management, Booking Management, personalised plans and scheduling.



Inventory & Equipment

Inventory Management, Equipment Management, Maintenance Management, supplier tracking, and stock alerts.

Each module is designed as an independently deployable unit with clear domain boundaries, following SOLID and DRY principles. This granular decomposition allows development teams to work in parallel, facilitates targeted regression testing, and supports incremental feature delivery throughout the project lifecycle.



Membership & Attendance

Membership Management, Attendance Management, QR-based check-in, renewals, and membership lifecycle tracking.



Finance & Billing

Payment Management, Billing & Invoicing, inventory costing, supplier payments, and financial reporting integration.



Insights & Config

Dashboard & Analytics, Reports, Notifications, Settings, and a future AI Assistant for intelligent recommendations.

Required Diagrams

Six categories of UML and database diagrams document the system from multiple angles. Each diagram type serves a distinct modelling purpose — from capturing actor interactions to mapping deployment topology.



Together these diagrams provide a complete visual specification that bridges requirements and implementation, ensuring all stakeholders share a common understanding of system structure, behaviour, and deployment topology before development commences.

User Interface Guidelines

The GymX interface adheres to modern UI/UX principles designed for clarity, efficiency, and cross-device responsiveness. A consistent design language spans every module, reducing the learning curve for administrators, staff, and members alike.

Design Principles

- Responsive design across desktop, tablet, and mobile
- Light & Dark mode with system-aware switching
- Consistent, accessible colour palette
- Easy navigation with persistent sidebar
- Dashboard cards for at-a-glance metrics
- Interactive charts for trends and analytics
- Data tables with search, filters, and pagination
- Form validation with real-time feedback
- Toast notifications for action confirmations

Recommended Colour Palette

Role	Hex Code	Colour
Primary	#2563EB	Blue
Success	#16A34A	Green
Warning	#F59E0B	Orange
Danger	#DC2626	Red
Background	#FFFFFF / #F3F4F6	White / Light Gray

Semantic colour usage ensures that interface states are immediately recognisable. Blue conveys primary actions and navigation, green confirms successful operations, orange signals caution, and red flags errors — a convention familiar to users across most modern web applications.

Coding Standards

Consistent coding conventions are enforced across both backend and frontend codebases. These standards promote readability, reduce onboarding time for new engineers, and minimise the risk of technical debt accumulating over the project lifecycle.



Backend

- Python PEP8 compliance
- Django best practices
- RESTful API design standards
- Modular architecture
- SOLID principles
- DRY (Don't Repeat Yourself)



Frontend

- Component-based design
- ESLint for code quality
- Clean, intuitive folder structure
- Reusable, composable components
- Consistent naming conventions
- Accessibility-first markup

Backend code adheres to PEP8 and Django's established conventions, with RESTful endpoint design following predictable resource-oriented patterns. SOLID and DRY principles guide module decomposition, keeping services testable and maintainable. On the frontend, a component-based architecture with ESLint enforcement ensures that the React codebase remains clean, modular, and accessible. Folder structure conventions are documented and enforced from project inception.

Testing Strategy

A multi-layered testing strategy ensures correctness at every level of the system, from individual functions through end-to-end user journeys. Each testing type targets a specific quality dimension and is supported by industry-standard tooling.

Testing Pyramid

- **Unit Testing** — Validates individual functions, classes, and modules in isolation.
- **Integration Testing** — Verifies that modules interact correctly across layer boundaries.
- **API Testing** — Exercises every REST endpoint with valid and invalid payloads.
- **System Testing** — Assesses the full application end-to-end in a staging environment.
- **Regression Testing** — Ensures new changes do not break existing functionality.
- **Performance Testing** — Measures response times and throughput under simulated load.
- **Security Testing** — Identifies vulnerabilities including injection, XSS, and auth flaws.
- **User Acceptance Testing** — Confirms the system meets business and user requirements.

Recommended Tools

PyTest

Python unit & integration tests

Postman

API contract testing & collections

Swagger

API documentation & exploration

Selenium

Browser-based system & UAT tests

Locust

Load & performance testing

Unit and integration tests form the broad base of the pyramid, running on every commit via CI pipelines. API tests validate endpoint contracts using Postman collections published from Swagger specifications. Performance testing with Locust identifies bottlenecks under realistic user load, simulating peak gym check-in and booking scenarios.

Deployment Strategy

The production deployment leverages containerised services orchestrated on Ubuntu Linux, with Nginx handling ingress and static asset delivery, Gunicorn serving the Django WSGI application, and Docker encapsulating each service for portability.

1	Operating System Ubuntu Linux — stable, well-supported, production-hardened.
2	Web Server Nginx — reverse proxy, TLS termination, and static file caching.
3	Application Server Gunicorn — WSGI HTTP server running multiple Django workers.
4	Backend Framework Django — powers the REST API and business logic layer.
5	Database PostgreSQL — primary relational store with managed backups.
6	Cache Redis — in-memory cache for sessions and frequently accessed data.
7	Containers Docker — encapsulates each service for reproducible deployments.
8	CI/CD GitHub Actions (planned) — automated build, test, and deploy pipelines.

Cloud hosting targets AWS, Azure, or DigitalOcean, giving the organisation flexibility to select the provider that best meets its compliance, cost, and geographic requirements. A future CI/CD pipeline via GitHub Actions will automate build, test, and deployment steps, reducing manual overhead and ensuring consistent releases.

Project Roadmap

The GymX development lifecycle is organised into eight sequential phases — from initial planning and documentation through to long-term maintenance — ensuring a structured, risk-managed delivery.



Phases 1 through 5 form the core build cycle, with each phase gated by review checkpoints. Phases 6 and 7 converge quality assurance with production readiness, whilst Phase 8 establishes an ongoing maintenance cadence that keeps the platform secure, performant, and aligned with evolving business needs.

Glossary, Appendices & Final Conclusion

GymX is designed as a scalable, secure, and enterprise-level Gym Management System capable of managing every facet of modern fitness centre operations. The architecture prioritises maintainability, security, performance, and future scalability, whilst delivering an excellent user experience for administrators, employees, coaches, and members.

Glossary

Term	Definition
API	Application Programming Interface
JWT	JSON Web Token
RBAC	Role-Based Access Control
CRUD	Create Read Update Delete
REST	Representational State Transfer
ORM	Object Relational Mapping
ERD	Entity Relationship Diagram
UML	Unified Modelling Language
KPI	Key Performance Indicator
SaaS	Software as a Service

Appendices

01	A – Database Schema Complete DDL specifications
02	B – API Documentation Swagger/OpenAPI specifications
03	C – UI Mockups Wireframes and high-fidelity designs
04	D-I – Diagrams ER, Use Case, Sequence, Class, Activity, Deployment

 This Software Requirements Specification serves as the official reference for analysis, design, development, testing, deployment, and future maintenance of the GymX platform.